

Aufgabenstellung: HabitApp (CLI) – Gewohnheiten tracken mit CSV

Version: 2026-03-01 | Kurs/Gruppe: _____ | Name: _____

Ziel

Entwickle eine HabitApp ohne grafische Benutzeroberfläche. Die Bedienung erfolgt ausschließlich über die Konsole (Command Line Interface, CLI). Alle Daten müssen dauerhaft in CSV-Dateien gespeichert werden (keine Datenbank). Die App soll das Anlegen und Verfolgen von guten und schlechten Gewohnheiten ermöglichen.

Rahmenbedingungen

- Programmiersprache: Python 3
- Keine GUI/kein Web-Frontend (nur Konsole)
- Persistenz ausschließlich über CSV-Dateien (kein SQLite, keine ORM)
- Erlaubt: csv, datetime; optional: pandas (für Auswertungen)
- Saubere Projektstruktur (z.B. cli.py, storage.py, models.py, stats.py) und verständliche Fehlerbehandlung

Fachliche Anforderungen (MUSS)

- 1 **Habits anlegen:** Name, Typ (good/bad), Zeitraum (daily/weekly), optional Frequenz (z.B. 3x/Woche), Startdatum.
- 2 **Habits anzeigen:** Liste aller aktiven Habits (mit Typ und Zeitraum).
- 3 **Check-in** (für ein Datum, Standard: heute): Ein Habit wird als Erfolg/Misserfolg erfasst.
- 4 **Skip:** Für ein Datum wird das Habit als "übersprungen" gespeichert.
- 5 **Deaktivieren/Löschen:** Habits können deaktiviert werden (empfohlen: active=false).
- 6 **Statistiken:** mindestens Erfolge, Fehlschläge, Skips und Streak.

Definition: Gute vs. schlechte Gewohnheit

Gute Gewohnheit (good): Erfolg, wenn die Aktivität durchgeführt wurde ("gemacht").

Schlechte Gewohnheit (bad): Erfolg, wenn die Aktivität vermieden wurde ("widerstanden").

CSV-Speicherung (Pflicht)

Es sind mindestens zwei CSV-Dateien zu verwenden: eine für Stammdaten (Habits) und eine für Log-Einträge (Tage/Wochen). Die Dateien müssen beim Programmstart eingelesen und bei Änderungen zuverlässig aktualisiert werden.

Datei A: habits.csv (Stammdaten)

Spalte	Typ/Beispiel	Beschreibung
habit_id	int/uuid	Eindeutige ID
name	str	Name der Gewohnheit (z.B. "Joggen")
type	good bad	Gute oder schlechte Gewohnheit
period	daily weekly	Täglich oder wöchentlich
frequency	int	Nur bei weekly: Zielhäufigkeit pro Woche (sonst 0/leer)
start_date	YYYY-MM-DD	Startdatum
active	true false	Aktiv oder deaktiviert

Datei B: logs.csv (Log-Einträge)

Spalte	Typ/Beispiel	Beschreibung
log_id	int/uuid	Eindeutige ID
habit_id	int/uuid	Verweis auf habits.csv
date	YYYY-MM-DD	Datum des Eintrags
status	success fail skip	Erfolg/Misserfolg/Übersprungen

Menüanforderung (Beispiel)

Implementiere ein wiederkehrendes Konsolen-Menü. Beispiel:

- 1) Habit anlegen
- 2) Habits anzeigen
- 3) Check-in (heute)
- 4) Check-in (Datum eingeben)
- 5) Skip (heute)
- 6) Habit deaktivieren
- 7) Statistiken anzeigen
- 0) Beenden

Streak-Regel (Pflicht)

Definiere und dokumentiere eindeutig, wann eine Streak gilt. Mindestanforderung:

- **Daily:** Streak = Anzahl aufeinanderfolgender Tage mit status=success.
- **Weekly:** Streak = Anzahl aufeinanderfolgender Wochen, in denen die Wochen-Frequenz erreicht wurde.
- Entscheide, ob **skip** eine Streak unterbricht oder nicht, und setze diese Regel konsequent um.

Abgabe

- Quellcode (Projektordner als ZIP oder Repo-Link)
- README.md (Startanleitung, Menübeschreibung, CSV-Struktur, getroffene Streak-Regel)
- Beispiel-CSV-Dateien mit Testdaten (habits.csv, logs.csv)

Bewertung (Vorschlag)

Kriterium	Punkte
Funktionalität (CRUD, Check-in, CSV korrekt)	40
Datenmodell & CSV-Design (sauber, robust, validiert)	20
Statistiken & Streak (korrekt, nachvollziehbar)	20
Codequalität (Struktur, Lesbarkeit, Fehlerhandling)	20
Summe	100

Hinweise

- Verwende eindeutige IDs (z.B. fortlaufend oder uuid4).
- Verhindere doppelte Log-Einträge: Pro Habit und Datum soll es höchstens einen Eintrag geben (Update statt Duplikat).
- Validiere Eingaben (Datumformat, gültige Menüauswahl, frequency nur bei weekly).